

JSON Schema

Validate JSON contracts and apply declared defaults



Install & load

```
INSTALL json_schema FROM community;  
LOAD json_schema;
```

01 At a glance

Check JSON documents against a schema inside SQL, then fill missing properties from schema defaults. Useful at ingest and during audits. Invalid documents raise errors in this build, so choose strict validation or a deliberate TRY wrapper.

Schema + JSON → Validate → Patch or enrich

02 Core SQL surface

`json_schema_validate(schema, data)`

Return TRUE on success; raise an error on invalid data.

`json_schema_validate_schema(schema)`

Check schema acceptance; invalid schemas can raise errors.

`json_schema_patch(schema, data)`

Return JSON Patch operations for missing defaults.

`json_schema_update(schema, data)`

Return the document with defaults applied.

`TRY(expression)`

DuckDB wrapper: convert runtime validation errors into NULL.

03 Keep in mind

Make validation policy explicit

Plain validation aborts on nonconforming input. `COALESCE(TRY(json_schema_validate(...)), false)` gives a filterable flag. Lint your schema first: TRY also hides malformed schemas, so a bad contract could otherwise mark every row invalid.

04 Flag valid and invalid rows

```
SELECT payload, coalesce(try(  
  json_schema_validate(  
    '{"type":"integer"}', payload  
  )), false) AS valid  
FROM (VALUES ('5'::JSON), ('five'::JSON))  
  t(payload);  
-- 5: true; "five": false
```

TRY preserves the query when a row violates the schema.

05 Fill a missing property

```
SET VARIABLE schema = '{  
  "type":"object",  
  "properties":{  
    "retries":{"type":"integer","default":3}  
  }  
}';  
SELECT json_schema_validate_schema(  
  getvariable('schema'));  
SELECT json_schema_update(  
  getvariable('schema'), '{}');  
-- {"retries":3}  
SELECT json_schema_patch(  
  getvariable('schema'), '{}');
```

The patch contains an add operation at /retries with value 3.

06 In practice

Draft and defaults

The validator primarily targets draft-07. Test the keywords your contract uses and keep reference targets within the schema. Defaults are applied by patch/update; a successful validate call alone does not rewrite the original JSON.

[Full docs & examples ↗](#) [Source ↗](#)